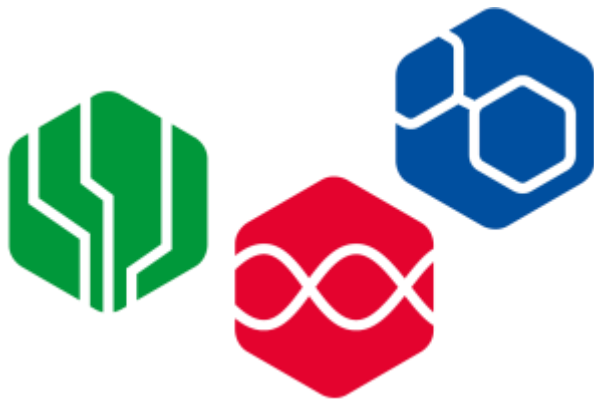


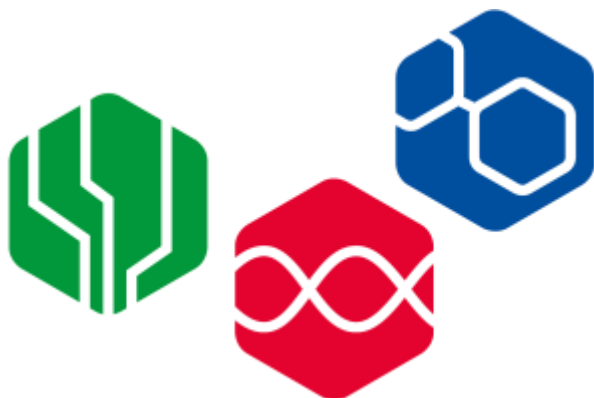
[Zum Hauptinhalt](#)



- [Startseite](#)
- [Dashboard](#)
- [Meine Kurse](#)
- [ITECH Website](#)
- [Lernortkooperation \(LOK\)](#)
- [4teacher](#)

[Sucheingabe umschalten](#)

1



-
-
-
-
-

-
-
-
-
-
-
-
-

1. [BHH@ITECH - Signale und Bussysteme](#)
2. [Arbeitsmaterialien](#)
3. Pulsweitenmodulation (PWM) zur Steuerung von Aktoren verwenden

Pulsweitenmodulation (PWM) zur Steuerung von Aktoren verwenden

Abschlussbedingungen

Warum Pulsweitenmodulation?

Zur Leistungssteuerung vieler Aktoren (beispielsweise Leuchtdioden oder Motoren)

wird die Pulsweitenmodulation verwendet.

Die Kennlinien vieler Aktoren sind steil, nicht-linear und abhängig von Umgebungsgrößen.

Es wird daher NICHT versucht, durch das Anlegen einer analogen Spannung den gewünschten Arbeitspunkt und damit die gewünschte Leistungsaufnahme zu erreichen.

Stattdessen wird die elektrische Spannung für den Aktor schnell ein und wieder ausgeschaltet. Das Verhältnis von ein- zu ausgeschaltetem Zustand entspricht der gewünschten Leistungsaufnahme.

Eine Pulsweite von 0% bedeutet: Der Aktor erhält keine Leistung, ist also ausgeschaltet.

Eine Pulsweite von 100% bedeutet: Der Aktor erhält die volle Leistung, ist also eingeschaltet.

Eine Pulsweite von 50%: Der Aktor ist die Hälfte der Zeit eingeschaltet und die andere Hälfte der Zeit ausgeschaltet und erhält so die halbe Leistung.

Für andere Pulsweiten geht die Berechnung analog.

Für den Arduino muss eine Pulsweitenmodulation nicht selbst (über Timer Interrupts) programmiert werden.

Stattdessen stellt die Arduino Plattform die Funktion `analogWrite()` zur Verfügung, die eine komfortable PWM-Funktion bietet.

Über einen Parameter (von 0 bis 255) wird die gewünschte prozentuale Pulsweite

übergeben.

Ein kleines Beispiel um eine LED (oder einen Motor) mit der halben Leistung zu betreiben.

```
int ledPin = 3;           // LED connected to digital pin 3
int val = 0;             // variable to store the read value

void setup() {
  pinMode(ledPin, OUTPUT); // sets the pin as output
}

void loop() {
  analogWrite(ledPin, 127);

  delay(300);           // has no impact! analogWrite() generates a steady rectangular
  // when started
}
```

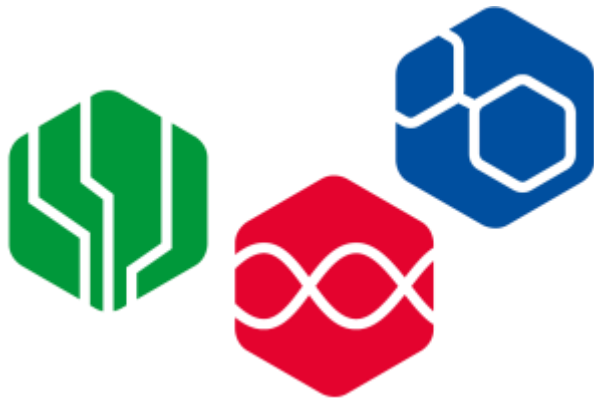
Hier eine [Anleitung für den Raspberry Pi](#)

Zuletzt geändert: Sonntag, 29. Mai 2022, 19:34

Schreibe ein kleines Programm, dass eine LED langsam von 0% Lichtstärke auf 100% Lichtstärke und anschließend wieder auf 0% Lichtstärke dimmt.
(wahlweise einen Motor langsam auf 100% Leistung anfahren lässt und anschließend wieder langsam anhalten lässt)

Zuletzt geändert: Freitag, 27. Mai 2022, 22:22

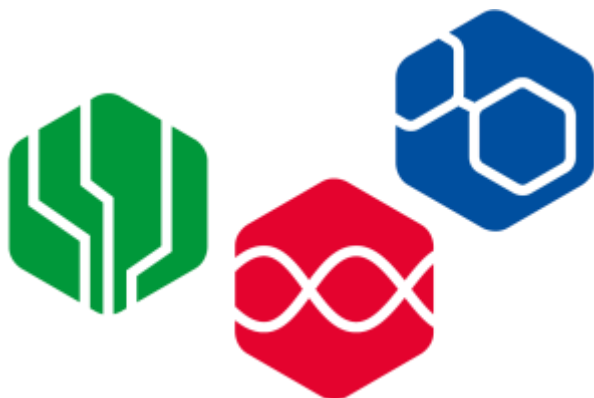
[Zum Hauptinhalt](#)



- [Startseite](#)
- [Dashboard](#)
- [Meine Kurse](#)
- [ITECH Website](#)
- [Lernortkooperation \(LOK\)](#)
- [4teacher](#)

[Sucheingabe umschalten](#)

1



-
-
-
-
-
-

-
-
-
-
-
-
-

1. [BHH@ITECH - Signale und Bussysteme](#)
2. [Arbeitsmaterialien](#)
3. I2C Bus zur Kommunikation nutzen

I2C Bus zur Kommunikation nutzen

Abschlussbedingungen

Für die Kommunikation einzelner Geräte oder Komponenten eines Gesamtsystems werde häufig Bus-Systeme eingesetzt.

An ein solches Bus-System können mehrere Geräte angeschlossen werden, die über individuelle Adressen einzeln angesprochen werden können.

Durch den Einsatz eines Bus-Systems spart man insbesondere bei mehreren Komponenten Hardwareanschlüsse, da nicht für jedes einzelne Gerät ein Anschluss-Pin (GPIO) zu Verfügung gestellt werden muss, sondern 1-2 Pins für das gesamte Bussystem.

Ein in Microcontrolleranwendungen häufig genutztes Bussystem ist der I2C (sprich *I-quadrat-C*)-Bus.

I2C (Inter-Integrated Circuit)

I2C (Inter-Integrated Circuit) ist ein On-Board-Kommunikationsprotokoll, das ideal für kurze Distanzen und geringe Bandbreite ist. Es hat eine Master-Slave-Architektur, in der alle Slaves über zwei Leitungen mit dem Master verbunden sind: die serielle Datenleitung (SDA) und die serielle Taktleitung (SCL).

SDA: Serial Data

SCL: Serial Clock

In der Regel befinden sich an einem I2C-Bus ein Master und bis zu 127 Slaves. Ein Multimasterbetrieb ist jedoch möglich.

Die beiden Busse SDA und SCL werden über Pull-up Widerstände mit der positiven Versorgungsspannung verbunden, so dass, wenn niemand sendet, die Leitungen auf das High-Potenzial (z.B. +5V) gelegt sind.

Wie wird kommuniziert?

Der Master kontrolliert das Bussystem!

- Der Master sendet Daten
- Der Master fordert Daten von einem Slave an
- Slaves werden nicht ohne Aufforderung aktiv!

Der I2C-Bus ist ein serielles Bussystem, d.h. die Daten werden seriell (nacheinander) übertragen.

Das I2C Protokoll

Der I2C-Rahmen wird durch das Start-Bit eingeleitet und durch das Stop-Bit beendet.

Nach dem Startbit sendet der Master die Adresse des gewünschten Slaves.

Im nachfolgenden W/R-Bit wird festgelegt, ob der Master Daten schreiben oder anfordern möchte:

W/R: 0 bedeutet: Der Master überträgt Daten an den Slave.

W/R: 1 bedeutet: Der Master fordert Daten vom Slave an.

Mit ACK=0 bestätigt der Slave den Empfang der vorangegangenen Information.

Der Versand des Datenbytes wird ebenfalls mit ACK=0 bestätigt.

Es können nun weitere Datenbytes folgen, die jeweils mit ACK=0 bestätigt werden. Mit einem Stop-Bit wird die Kommunikation abgeschlossen.

Datenübertragung

Für die Datenübertragung werden die beiden Leitungen SCL und SDA genutzt. So lange das Clock-Signal SCL auf High liegt, sind die Daten auf der SDA-Leitung gültig und können vom Empfänger ausgelesen werden. Ein Wechsel der Daten ist daher nur bei SCL auf Low möglich.

Wenn sich der Zustand von SDA ändert, wenn SCL auf Low ist, führt das zu einer Protokollverletzung. Das wird beim Start- und Stop-Bit genutzt.

Start- und Stop-Bit

Einen Arduino als Master mit I2C-Aktoren und -Sensoren oder weiteren Arduinos als Slave betreiben

SCL und SDA können zwar manuell über allgemeine GPIO-Ports betrieben werden. Um die Nutzung jedoch komfortabler zu machen, stellt die Arduino-Plattform spezielle I2C-Bibliotheken zur Verfügung.

Unter anderem ist die wire-Bibliothek sehr hilfreich, die [hier dokumentiert](#) ist.

Für manche Sensoren und Aktoren werden auch spezielle Bibliotheken zur Verfügung gestellt, die auch komplexere Aktionen durch einfache Funktionsaufrufe ermöglichen. Beschreibungen zu solchen Bibliotheken findet man ggf. in der Dokumentation der Peripherie-Bausteine.

Ein einfaches Beispiel: Kommunikation zwischen zwei Arduinos

Ein Master schaltet im Sekundentakt die Leuchtdiode D13 beim zweiten Arduino, dem Slave, um. Dazu sendet der Master einfach ein beliebiges Byte im Sekundentakt an den Slave. Wenn beim Slave ein Byte ankommt dreht er den Status der Leitung um.

Schaltung:

- die beiden GND-Leitungen der Arduinos verbinden
- die beiden A5- (SCL) und A4- (SDA) Leitungen verbinden
- irgendwo Widerstände (1k ... 10k) jeweils an die A5- bzw. A4-Leitung und die positive Versorgungsspannung legen (Pull-Up Widerstände)

Code Master

```
#include <Wire.h>

void setup()
{
  Wire.begin();
}

void loop()
{
  Wire.beginTransmission(42);

  Wire.write(0);

  boolean allesgut = Wire.endTransmission();

  delay(1000);
}
```

Im Setup() wird der Bus initialisiert und in der Loop() jeweils die Übertragung durchgeführt - beginnen, senden, beenden, warten, wiederholen.

Die Daten werden mit write in einem 32 Byte großen Puffer gespeichert. Die tatsächliche Übertragung findet erst mit dem .endTransmission-Aufruf statt, .

Über den Rückgabewert von .endTransmission kann kontrolliert werden, ob der Slave irgendwie die Daten angenommen hat. Es ist keine Kontrolle, ob alle Daten fehlerfrei angekommen sind.

Code Slave

```
#include <Wire.h>

void setup()
{
  pinMode(13,OUTPUT);
  Wire.begin(42);
}
```

```

    Wire.onReceive(receiveEvent);
}

void loop()
{
    //nix.
}

void receiveEvent(int anzahl)
{
    while(Wire.available())
    {
        Wire.read();
    }

    digitalWrite(13,! digitalRead(13));
}

```

Die Wire-Umgebung ruft, sobald Daten vorliegen, die Empfangsroutine selbsttätig auf. Das ist mit den Interrupts (attachInterrupt) vergleichbar.

Im Setup wird die Funktion dem .onReceive-Ereignis zugewiesen, weiter unten wird die Funktion dann deklariert. Wichtig ist, dass der Datentyp des Übergabeparameters passt: hier int. Das kann bei andere Wire-Routinen für andere Prozessoren anders sein.

Wenn Daten ankommen, ruft die Wire-Bibliothek die receiveEvent()-Funktion auf, in der die Daten gelesen werden können. Oft wird die übergebene Anzahl ignoriert und einfach so lange gelesen, bis keine Daten mehr verfügbar sind.

In diesem Beispiel werden die Daten nicht ausgewertet. Stattdessen wird LED geschaltet.

Arduino und Raspberry Pi über den I2C Bus verbinden

Da der Arduino und der Raspberry Pi mit unterschiedlichen Spannungsebenen arbeiten (Arduino 5 V, Raspberry Pi 3,3V) müssen diese Unterschiede bei der Anbindung der beiden Systeme beachtet werden.

Unproblematisch ist hierbei, den Raspberry Pi als Master zu betreiben und die Pullup-Widerstände mit der Raspberry Pi Betriebsspannung $U=3,3V$ zu bringen.

(Die Verwendung der Arduino Betriebsspannung von $U=5V$ kann die Eingangsports des Raspberry Pi zerstören)

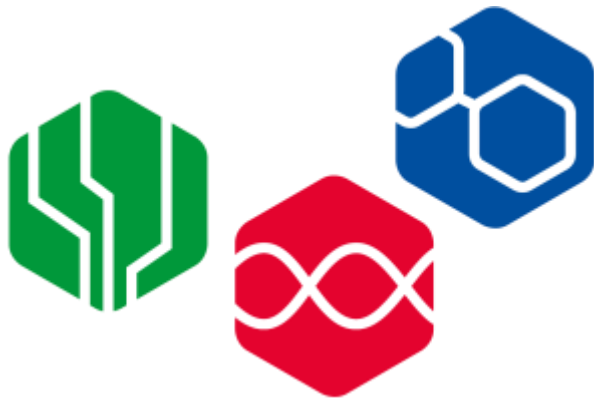
Eine Erklärung zu den Hintergründen findet man u.a. auch in diesem Youtube-Video:



Video abspielen

Zuletzt geändert: Montag, 10. Juni 2024, 21:32

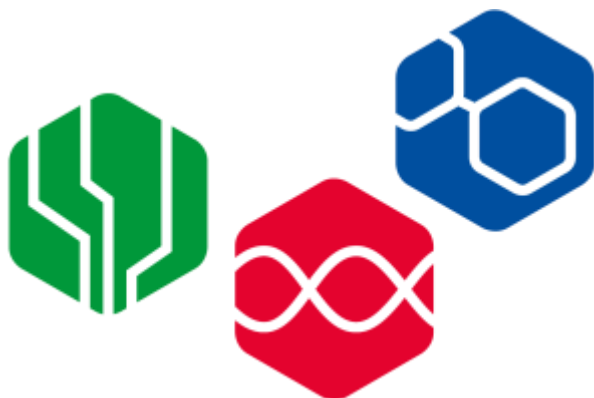
[Zum Hauptinhalt](#)



- [Startseite](#)
- [Dashboard](#)
- [Meine Kurse](#)
- [ITECH Website](#)
- [Lernortkooperation \(LOK\)](#)
- [4teacher](#)

[Sucheingabe umschalten](#)

1



-
-
-
-
-
-

-
-
-
-
-
-
-

1. [BHH@ITECH - Signale und Bussysteme](#)
2. [Arbeitsmaterialien](#)
3. I2C Bus zur Kommunikation nutzen

I2C Bus zur Kommunikation nutzen

Abschlussbedingungen

Für die Kommunikation einzelner Geräte oder Komponenten eines Gesamtsystems werde häufig Bus-Systeme eingesetzt.

An ein solches Bus-System können mehrere Geräte angeschlossen werden, die über individuelle Adressen einzeln angesprochen werden können.

Durch den Einsatz eines Bus-Systems spart man insbesondere bei mehreren Komponenten Hardwareanschlüsse, da nicht für jedes einzelne Gerät ein Anschluss-Pin (GPIO) zu Verfügung gestellt werden muss, sondern 1-2 Pins für das gesamte Bussystem.

Ein in Microcontrolleranwendungen häufig genutztes Bussystem ist der I2C (sprich *I-quadrat-C*)-Bus.

I2C (Inter-Integrated Circuit)

I2C (Inter-Integrated Circuit) ist ein On-Board-Kommunikationsprotokoll, das ideal für kurze Distanzen und geringe Bandbreite ist. Es hat eine Master-Slave-Architektur, in der alle Slaves über zwei Leitungen mit dem Master verbunden sind: die serielle Datenleitung (SDA) und die serielle Taktleitung (SCL).

SDA: Serial Data

SCL: Serial Clock

In der Regel befinden sich an einem I2C-Bus ein Master und bis zu 127 Slaves. Ein Multimasterbetrieb ist jedoch möglich.

Die beiden Busse SDA und SCL werden über Pull-up Widerstände mit der positiven Versorgungsspannung verbunden, so dass, wenn niemand sendet, die Leitungen auf das High-Potenzial (z.B. +5V) gelegt sind.

Wie wird kommuniziert?

Der Master kontrolliert das Bussystem!

- Der Master sendet Daten
- Der Master fordert Daten von einem Slave an
- Slaves werden nicht ohne Aufforderung aktiv!

Der I2C-Bus ist ein serielles Bussystem, d.h. die Daten werden seriell (nacheinander) übertragen.

Das I2C Protokoll

Der I2C-Rahmen wird durch das Start-Bit eingeleitet und durch das Stop-Bit beendet.

Nach dem Startbit sendet der Master die Adresse des gewünschten Slaves.

Im nachfolgenden W/R-Bit wird festgelegt, ob der Master Daten schreiben oder anfordern möchte:

W/R: 0 bedeutet: Der Master überträgt Daten an den Slave.

W/R: 1 bedeutet: Der Master fordert Daten vom Slave an.

Mit ACK=0 bestätigt der Slave den Empfang der vorangegangenen Information.

Der Versand des Datenbytes wird ebenfalls mit ACK=0 bestätigt.

Es können nun weitere Datenbytes folgen, die jeweils mit ACK=0 bestätigt werden. Mit einem Stop-Bit wird die Kommunikation abgeschlossen.

Datenübertragung

Für die Datenübertragung werden die beiden Leitungen SCL und SDA genutzt. So lange das Clock-Signal SCL auf High liegt, sind die Daten auf der SDA-Leitung gültig und können vom Empfänger ausgelesen werden. Ein Wechsel der Daten ist daher nur bei SCL auf Low möglich.

Wenn sich der Zustand von SDA ändert, wenn SCL auf Low ist, führt das zu einer Protokollverletzung. Das wird beim Start- und Stop-Bit genutzt.

Start- und Stop-Bit

Einen Arduino als Master mit I2C-Aktoren und -Sensoren oder weiteren Arduinos als Slave betreiben

SCL und SDA können zwar manuell über allgemeine GPIO-Ports betrieben werden. Um die Nutzung jedoch komfortabler zu machen, stellt die Arduino-Plattform spezielle I2C-Bibliotheken zur Verfügung.

Unter anderem ist die wire-Bibliothek sehr hilfreich, die [hier dokumentiert](#) ist.

Für manche Sensoren und Aktoren werden auch spezielle Bibliotheken zur Verfügung gestellt, die auch komplexere Aktionen durch einfache Funktionsaufrufe ermöglichen. Beschreibungen zu solchen Bibliotheken findet man ggf. in der Dokumentation der Peripherie-Bausteine.

Ein einfaches Beispiel: Kommunikation zwischen zwei Arduinos

Ein Master schaltet im Sekundentakt die Leuchtdiode D13 beim zweiten Arduino, dem Slave, um. Dazu sendet der Master einfach ein beliebiges Byte im Sekundentakt an den Slave. Wenn beim Slave ein Byte ankommt dreht er den Status der Leitung um.

Schaltung:

- die beiden GND-Leitungen der Arduinos verbinden
- die beiden A5- (SCL) und A4- (SDA) Leitungen verbinden
- irgendwo Widerstände (1k ... 10k) jeweils an die A5- bzw. A4-Leitung und die positive Versorgungsspannung legen (Pull-Up Widerstände)

Code Master

```
#include <Wire.h>

void setup()
{
  Wire.begin();
}

void loop()
{
  Wire.beginTransmission(42);

  Wire.write(0);

  boolean allesgut = Wire.endTransmission();

  delay(1000);
}
```

Im Setup() wird der Bus initialisiert und in der Loop() jeweils die Übertragung durchgeführt - beginnen, senden, beenden, warten, wiederholen.

Die Daten werden mit write in einem 32 Byte großen Puffer gespeichert. Die tatsächliche Übertragung findet erst mit dem .endTransmission-Aufruf statt, .

Über den Rückgabewert von .endTransmission kann kontrolliert werden, ob der Slave irgendwie die Daten angenommen hat. Es ist keine Kontrolle, ob alle Daten fehlerfrei angekommen sind.

Code Slave

```
#include <Wire.h>

void setup()
{
  pinMode(13,OUTPUT);
  Wire.begin(42);
}
```

```

    Wire.onReceive(receiveEvent);
}

void loop()
{
    //nix.
}

void receiveEvent(int anzahl)
{
    while(Wire.available())
    {
        Wire.read();
    }

    digitalWrite(13,! digitalRead(13));
}

```

Die Wire-Umgebung ruft, sobald Daten vorliegen, die Empfangsroutine selbsttätig auf. Das ist mit den Interrupts (attachInterrupt) vergleichbar.

Im Setup wird die Funktion dem .onReceive-Ereignis zugewiesen, weiter unten wird die Funktion dann deklariert. Wichtig ist, dass der Datentyp des Übergabeparameters passt: hier int. Das kann bei andere Wire-Routinen für andere Prozessoren anders sein.

Wenn Daten ankommen, ruft die Wire-Bibliothek die receiveEvent()-Funktion auf, in der die Daten gelesen werden können. Oft wird die übergebene Anzahl ignoriert und einfach so lange gelesen, bis keine Daten mehr verfügbar sind.

In diesem Beispiel werden die Daten nicht ausgewertet. Stattdessen wird LED geschaltet.

Arduino und Raspberry Pi über den I2C Bus verbinden

Da der Arduino und der Raspberry Pi mit unterschiedlichen Spannungsebenen arbeiten (Arduino 5 V, Raspberry Pi 3,3V) müssen diese Unterschiede bei der Anbindung der beiden Systeme beachtet werden.

Unproblematisch ist hierbei, den Raspberry Pi als Master zu betreiben und die Pullup-Widerstände mit der Raspberry Pi Betriebsspannung $U=3,3V$ zu bringen.

(Die Verwendung der Arduino Betriebsspannung von $U=5V$ kann die Eingangsports des Raspberry Pi zerstören)

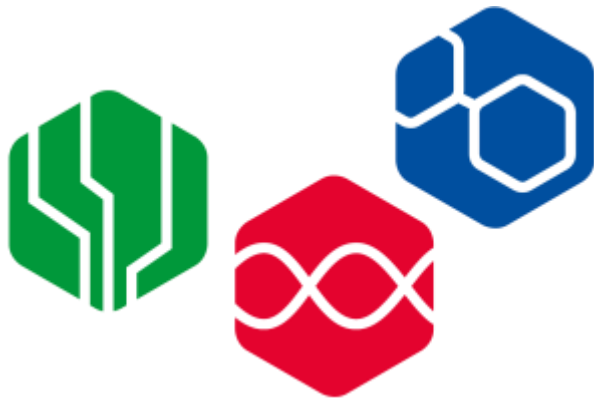
Eine Erklärung zu den Hintergründen findet man u.a. auch in diesem Youtube-Video:



Video abspielen

Zuletzt geändert: Montag, 10. Juni 2024, 21:32

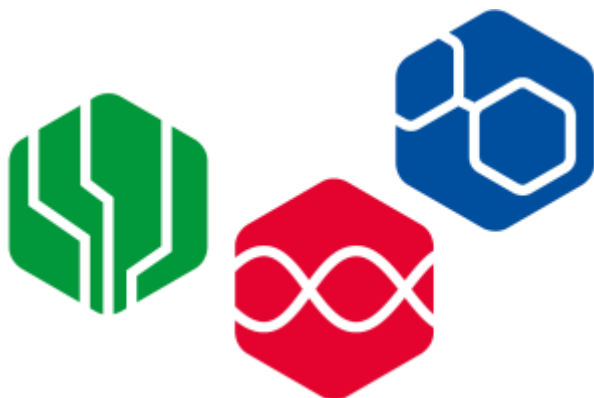
[Zum Hauptinhalt](#)



- [Startseite](#)
- [Dashboard](#)
- [Meine Kurse](#)
- [ITECH Website](#)
- [Lernortkooperation \(LOK\)](#)
- [4teacher](#)

[Sucheingabe umschalten](#)

1



-
-
-
-
-
-

-
-
-
-
-
-
-

1. [BHH@ITECH - Signale und Bussysteme](#)
2. [Arbeitsmaterialien](#)
3. Übungen zu I2C

Übungen zu I2C

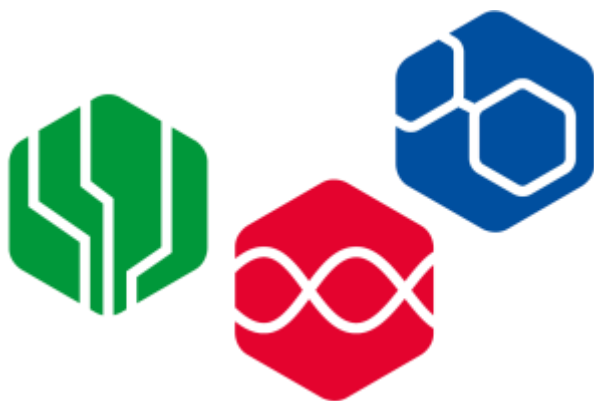
Abschlussbedingungen

Übungen zu I2C

- Welche I2C-Sensoren und -Aktoren findest Du in Deinem Starterset?
Schau in die Dokumentation und implementiere die Beispielschaltungen.
- Schalte über den Raspberry Pi die LED des Arduinos.
- Experimentiere mit eigenen Schaltungen

Zuletzt geändert: Samstag, 28. Mai 2022, 21:06

[Zum Hauptinhalt](#)



- [Startseite](#)

1. [BHH@ITECH - Signale und Bussysteme](#)
2. [Arbeitsmaterialien](#)
3. Servomotoren ansteuern

Servomotoren ansteuern

Abschlussbedingungen

Servomotoren bestehen aus Gleichstrommotoren mit integrierter Regelkreiselektronik.

Durch ein externes Signal, z.B. von einem Mikrocontroller eines Arduinoboards, kann beispielsweise der Sollwinkel des Motors vorgegeben werden.

Die Regelkreiselektronik sorgt dann dafür, dass der Servomotor diesen Sollwinkel annimmt.

Anschlüsse des Servomotors

Der Servomotor verfügt über 3 Anschlüsse.

- Rot: positive Betriebsspannung, z.B. +5V
- Schwarz: GND
- Gelb: Signal

Ansteuerung des

Servomotors

Der Servomotor erwartet alle 20 Millisekunden (0,02 s) einen Impuls.

Die Länge des Impulses (von 1,25ms bis 1,75ms) gibt den Sollwinkel an.

Die Zuordnung der Winkel zu den Impulszeiten variiert zwischen den einzelnen Servomotoren.

Hier hilft ein Blick ins Datenblatt, bzw. Ausprobieren.

Nebenstehend ein mögliches Beispiel.

Auch für Servomotoren stellt die Arduino-Plattform eine Bibliothek zur Verfügung. Unter diesem folgendem Link findet man [Informationen zur Bibliothek servo.h](#).

Beispiel

Ein einfaches Beispiel zur Ansteuerung eines Servomotors:

```
#include <Servo.h>

Servo servoMain; // Define our Servo

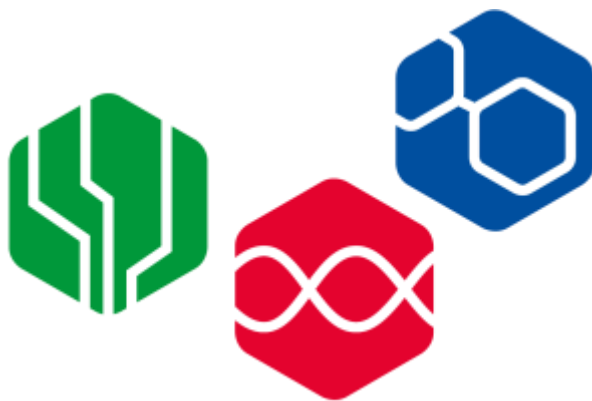
void setup()
{
  servoMain.attach(10); // servo on digital pin 10
}

void loop()
{
  servoMain.write(0); // Turn Servo Left to 45 degrees
  delay(1000); // Wait 1 second
```

```
servoMain.write(45); // Turn Servo Left to 0 degrees  
delay(1000); // Wait 1 second  
servoMain.write(90); // Turn Servo back to center position (90 degrees)  
delay(1000); // Wait 1 second  
servoMain.write(135); // Turn Servo Right to 135 degrees  
delay(1000); // Wait 1 second  
servoMain.write(180); // Turn Servo Right to 180 degrees  
delay(1000); // Wait 1 second  
servoMain.write(90); // Turn Servo back to center position (90 degrees)  
delay(1000); // Wait 1 second  
}
```

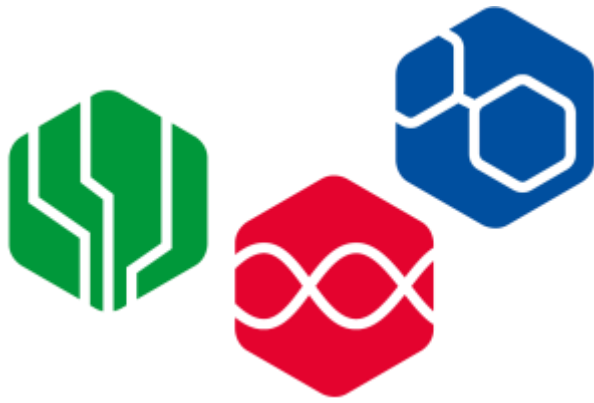
Zuletzt geändert: Sonntag, 29. Mai 2022, 16:14

[Zum Hauptinhalt](#)



- [Startseite](#)
- [Dashboard](#)
- [Meine Kurse](#)
- [ITECH Website](#)
- [Lernortkooperation \(LOK\)](#)
- [4teacher](#)

[Sucheingabe umschalten](#)



-
-
-
-
-
-
-
-
-
-
-
-
-
-

1. [BHH@ITECH - Signale und Bussysteme](#)
2. [Arbeitsmaterialien](#)
3. Einen Gleichstrommotor ansteuern

Einen Gleichstrommotor ansteuern

Abschlussbedingungen

Größere elektrische Verbraucher, wie beispielsweise ein Gleichstrommotor, benötigen eine höhere elektrische Leistung als ein Mikrocontroller an seinen GPIOs liefern kann.

Die Grenzen bei einem Arduino Uno liegen bei 40mA für alle GPIOs, bzw. 20mA pro GPIO. Selbst kleinere Gleichstrommotoren benötigen jedoch Stromstärken von 150-200mA.

Eine Lösung ist es, den Steueranschluss (Basis) eines Transistors über einen GPIO anzusteuern.

Hierdurch wird die Verbindung zwischen Kollektor und Emitter leitend und ein angeschlossener Motor kann über eine zusätzliche Spannungsquelle mit größeren Stromstärken versorgt werden. (siehe nebenstehendes Schaltbeispiel).

Die Größe des Basisvorwiderstands R_B ist abhängig vom Transistor. Ein Widerstandswert von $R_B=470\text{ Ohm}$ sollte aber funktionieren.

Um den Transistor zu schützen empfiehlt es sich, falls vorhanden, antiparallel zum Motor eine Freilaufdiode zu schalten.

Motortreiber L293D

Zur
Ansteuerung von Gleichstrommotoren gibt es spezielle Motortreiber. Der Baustein L293D ist in vielen Arduinosets enthalten.

Der L293D realisiert eine sogenannte H-Brücke, mit der ein Gleichstrommotor in beiden Drehrichtungen betrieben werden kann.

Über einen zusätzlichen Eingang kann über ein PWM-Signal die Drehgeschwindigkeit eingestellt werden. Freilaufdioden zum Schutz der integrierten Schaltung sind bereits verbaut (das D steht für Diode).

In folgendem Tutorial wird die [Funktionsweise und die Inbetriebnahme des Motortreibers L293D erklärt](#).

Zuletzt geändert: Sonntag, 29. Mai 2022, 17:11

Die große Stärke von "smarten Systemen" ist ihre Fähigkeit miteinander zu kommunizieren.

Sensoren melden Messergebnisse an Microcontroller. Diese geben diese Daten an zentrale Einheiten zur Auswertung und Verarbeitung weiter und senden Steuersignale an Aktoren, die wiederum Einfluss auf die Umwelt nehmen.

In diesem Fachmodul findest Du zunächst einen Einstieg in üblicherweise im Bereich IoT verwendeten Signale, wie die Pulsweitenmodulation und dem häufig in lokalen Umgebungen verwendeten Bussystem I2C.